

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

TOPIC 3.1

Abstraction and Program Design

Required Course Content

LEARNING OBJECTIVE

3.1.A

Represent the design of a program by using natural language or creating diagrams that indicate the classes in the program and the data and procedural abstractions found in each class by including all attributes and behaviors.

ESSENTIAL KNOWLEDGE

3.1.A.1

Abstraction is the process of reducing complexity by focusing on the main idea. By hiding details irrelevant to the question at hand and bringing together related and useful details, abstraction reduces complexity and allows one to focus on the idea.

3.1.A.2

Data abstraction provides a separation between the abstract properties of a data type and the concrete details of its representation. Data abstraction manages complexity by giving data a name without referencing the specific details of the representation. Data can take the form of a single variable or a collection of data, such as in a class or a set of data.

3.1.A.3

An *attribute* is a type of data abstraction that is defined in a class outside any method or constructor. An *instance variable* is an attribute whose value is unique to each instance of the class. A *class variable* is an attribute shared by all instances of the class.

3.1.A.4

Procedural abstraction provides a name for a process and allows a method to be used only knowing what it does, not how it does it. Through *method decomposition*, a programmer breaks down larger behaviors of the class into smaller behaviors by creating methods to represent each individual smaller behavior. A procedural abstraction may extract shared features to generalize functionality instead of duplicating code. This allows for code reuse, which helps manage complexity.

continued on next page

LEARNING OBJECTIVE

3.1.A

Represent the design of a program by creating diagrams that indicate the classes in the program and the data and procedural abstractions found in each class by including all attributes and behaviors.

ESSENTIAL KNOWLEDGE

3.1.A.5

Using parameters allows procedures to be generalized, enabling the procedures to be reused with a range of input values or arguments.

3.1.A.6

Using procedural abstraction in a program allows programmers to change the internals of a method (to make it faster, more efficient, use less storage, etc.) without needing to notify method users of the change as long as the method signature and what the method does is preserved.

3.1.A.7

Prior to implementing a class, it is helpful to take time to design each class including its attributes and behaviors. This design can be represented using natural language or diagrams.

SUGGESTED SKILLS

5.A

Explain how computing impacts society, economy, and culture.

TOPIC 3.2

Impact of Program Design

Required Course Content

LEARNING OBJECTIVE**3.2.A**

Explain the social and ethical implications of computing systems.

ESSENTIAL KNOWLEDGE**3.2.A.1**

System reliability refers to the program being able to perform its tasks as expected under stated conditions without failure. Programmers should make an effort to maximize system reliability by testing the program with a variety of conditions.

3.2.A.2

The creation of programs has impacts on society, the economy, and culture. These impacts can be both beneficial and harmful. Programs meant to fill a need or solve a problem can have unintended harmful effects beyond their intended use.

3.2.A.3

Legal issues and intellectual property concerns arise when creating programs. Programmers often reuse code written by others and published as open source and free to use. Incorporation of code that is not published as open source requires the programmer to obtain permission and often purchase the code before integrating it into their program.

TOPIC 3.3

Anatomy of a Class

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

2.B

Write program code involving data abstractions.

Required Course Content

LEARNING OBJECTIVE

3.3.A

Develop code to designate access and visibility constraints to classes, data, constructors, and methods.

ESSENTIAL KNOWLEDGE

3.3.A.1

Data encapsulation is a technique in which the implementation details of a class are kept hidden from external classes. The keywords `public` and `private` affect the access of classes, data, constructors, and methods. The keyword `private` restricts access to the declaring class, while the keyword `public` allows access from classes outside the declaring class.

3.3.A.2

In this course, classes are always designated `public` and are declared with the keyword `class`.

3.3.A.3

In this course, constructors are always designated `public`.

3.3.A.4

Instance variables belong to the object, and each object has its own copy of the variable.

3.3.A.5

Access to attributes should be kept internal to the class in order to accomplish encapsulation. Therefore, it is good programming practice to designate the instance variables for these attributes as `private` unless the class specification states otherwise.

3.3.A.6

Access to behaviors can be internal or external to the class. Methods designated as `public` can be accessed internally or externally to a class, whereas methods designated as `private` can only be accessed internally to the class.

SUGGESTED SKILLS

2.B

Write program code involving data abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

TOPIC 3.4

Constructors

Required Course Content

LEARNING OBJECTIVE

3.4.A

Develop code to declare instance variables for the attributes to be initialized in the body of the constructors of a class.

ESSENTIAL KNOWLEDGE

3.4.A.1

An object's *state* refers to its attributes and their values at a given time and is defined by instance variables belonging to the object. This defines a *has-a* relationship between the object and its instance variables.

3.4.A.2

A constructor is used to set the initial state of an object, which should include initial values for all instance variables. When a constructor is called, memory is allocated for the object and the associated object reference is returned. Constructor parameters, if specified, provide data to initialize instance variables.

3.4.A.3

When a mutable object is a constructor parameter, the instance variable should be initialized with a copy of the referenced object. In this way, the instance variable does not hold a reference to the original object, and methods are prevented from modifying the state of the original object.

3.4.A.4

When no constructor is written, Java provides a no-parameter constructor, and the instance variables are set to default values according to the data type of the attribute. This constructor is called the *default constructor*.

3.4.A.5

The default value for an attribute of type `int` is `0`. The default value of an attribute of type `double` is `0.0`. The default value of an attribute of type `boolean` is `false`. The default value of a reference type is `null`.

TOPIC 3.5

Methods: How to Write Them

Required Course Content

LEARNING OBJECTIVE

3.5.A

Develop code to define behaviors of an object through methods written in a class using primitive values and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

3.5.A.1

A `void` method does not return a value. Its header contains the keyword `void` before the method name.

3.5.A.2

A non-void method returns a single value. Its header includes the return type in place of the keyword `void`.

3.5.A.3

In non-void methods, a return expression compatible with the return type is evaluated, and the value is returned. This is referred to as *return by value*.

3.5.A.4

The `return` keyword is used to return the flow of control to the point where the method or constructor was called. Any code that is sequentially after a return statement will never be executed. Executing a return statement inside a selection or iteration statement will halt the statement and exit the method or constructor.

3.5.A.5

An *accessor method* allows objects of other classes to obtain a copy of the value of instance variables or class variables. An accessor method is a non-void method.

3.5.A.6

A *mutator (modifier) method* is a method that changes the values of the instance variables or class variables. A mutator method is often a void method.

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

continued on next page

LEARNING OBJECTIVE

3.5.A

Develop code to define behaviors of an object through methods written in a class using primitive values and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE

3.5.A.7

Methods with parameters receive values through those parameters and use those values in accomplishing the method's task.

3.5.A.8

When an argument is a primitive value, the parameter is initialized with a copy of that value. Changes to the parameter have no effect on the corresponding argument.

TOPIC 3.6

Methods: Passing and Returning References of an Object

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE**3.6.A**

Develop code to define behaviors of an object through methods written in a class using object references and determine the result of calling these methods.

ESSENTIAL KNOWLEDGE**3.6.A.1**

When an argument is an object reference, the parameter is initialized with a copy of that reference; it does not create a new independent copy of the object. If the parameter refers to a mutable object, the method or constructor can use this reference to alter the state of the object. It is good programming practice to not modify mutable objects that are passed as parameters unless required in the specification.

3.6.A.2

When the return expression evaluates to an object reference, the reference is returned, not a reference to a new copy of the object.

3.6.A.3

Methods cannot access the private data and methods of a parameter that holds a reference to an object unless the parameter is the same type as the method's enclosing class.

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.B

Determine the result or output based on code that contains data abstractions.

TOPIC 3.7

Class Variables and Methods

Required Course Content

LEARNING OBJECTIVE

3.7.A

Develop code to define behaviors of a class through class methods.

3.7.B

Develop code to declare the class variables that belong to the class.

ESSENTIAL KNOWLEDGE

3.7.A.1

Class methods cannot access or change the values of instance variables or call instance methods without being passed an instance of the class via a parameter.

3.7.A.2

Class methods can access or change the values of class variables and can call other class methods.

3.7.B.1

Class variables belong to the class, with all objects of a class sharing a single copy of the class variable. Class variables are designated with the `static` keyword before the variable type.

3.7.B.2

Class variables that are designated `public` are accessed outside of the class by using the class name and the dot operator, since they are associated with a class, not objects of a class.

3.7.B.3

When a variable is declared `final`, its value cannot be modified.

TOPIC 3.8

Scope and Access

SUGGESTED SKILLS

3.B

Determine the result or output based on code that contains data abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE**3.8.A**

Explain where variables can be used in the code.

ESSENTIAL KNOWLEDGE**3.8.A.1**

Local variables are variables declared in the headers or bodies of blocks of code. Local variables can only be accessed in the block in which they are declared. Since constructors and methods are blocks of code, parameters to constructors or methods are also considered local variables. These variables may only be used within the constructor or method and cannot be declared to be `public` or `private`.

3.8.A.2

When there is a local variable or parameter with the same name as an instance variable, the variable name will refer to the local variable instead of the instance variable within the body of the constructor or method.

SUGGESTED SKILLS

2.B

Write program code involving procedural abstractions.

TOPIC 3.9

this Keyword

Required Course Content

LEARNING OBJECTIVE

3.9.A

Develop code for expressions that are self-referencing and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

3.9.A.1

Within an instance method or a constructor, the keyword `this` acts as a special variable that holds a reference to the current object—the object whose method or constructor is being called.

3.9.A.2

The keyword `this` can be used to pass the current object as an argument in a method call.

3.9.A.3

Class methods do not have a `this` reference.